ASSIGNMENT – 2

JAVA STRINGS AND COLLECTION FRAMEWORK

Course Outcome: CO2

Maximum Marks: 50

Submitted by

Name: G. Sirisha

Register Number: 192472170

Academic Year: 2024

Submitted to

Faculty Name: Vijayaragavan

Submission Date: 03-11-26

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

Step 1: Aim

To compare Java String, StringBuilder, and StringBuffer while generating railway e-tickets under different workloads by measuring execution time, object creation and memory usage.

Step 2: Concept

1. String

- String objects are immutable.
- Every modification creates a new String object.
- Repeated concatenation using + creates unnecessary intermediate objects.

2. StringBuilder

- Mutable sequence of characters.
- Does not create a new object for every append operation.

- Faster for repeated modifications in single-threaded applications.
- Not synchronized.

3. StringBuffer

- Mutable sequence of characters.
- Its methods are synchronized.
- Suitable when multiple threads access the same buffer.
- Usually slower than StringBuilder because of synchronization overhead.

Step 3: Prediction Before Implementation

Approach	**Expected behaviour**
String (+)	Slow for large workloads
StringBuilder	Fastest in typical single-threaded workloads
StringBuffer	Slightly slower than StringBuilder

Reason:

String is immutable, so repeated concatenation creates new objects and copies existing characters. StringBuilder and StringBuffer modify their internal character storage, reducing unnecessary object creation.

Step 4: Design Three Workloads

We will test three different workloads.

Workload	**Tickets**	**Fields per ticket**	**Purpose**
A	1	6	Small workload
B	10,000	50	Large workload
C	1,000	100	Many fields per ticket

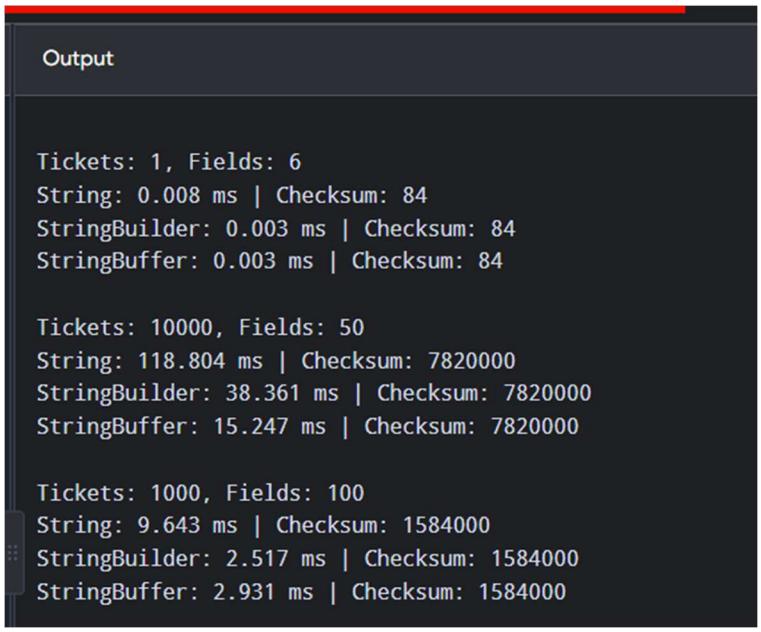
These workloads help identify how performance changes when the number of tickets and fields increases.

Step 5: Java Implementation

```
public class TicketBenchmark {
    static String buildTicket(int fields, int type) {
        String result = "";
        StringBuilder sb = new StringBuilder();
        StringBuffer buffer = new StringBuffer();
        for (int i = 1; i <= fields; i++) {
            String value = "Field" + i + ":Value" + i + " ";
            if (type == 1) {
                result += value;
            } else if (type == 2) {
                sb.append(value);
            } else {
                buffer.append(value);
            }
        }
        if (type == 1) return result;
        if (type == 2) return sb.toString();
        return buffer.toString();
    }
    static long execute(int tickets, int fields, int type) {
        long checksum = 0;
        for (int i = 0; i < tickets; i++) {
```

```
            String ticket = buildTicket(fields, type);
            checksum += ticket.length();
        }
        return checksum;
    }
    static void test(int tickets, int fields) {
        String[] methods = {
            "String", "StringBuilder", "StringBuffer"
        };
        System.out.println("\nTickets: " + tickets +
                           ", Fields: " + fields);
        for (int type = 1; type <= 3; type++) {
            execute(200, fields, type); // Warm-up
            long total = 0;
            long checksum = 0;
            for (int r = 0; r < 5; r++) {
                long start = System.nanoTime();
                checksum = execute(tickets, fields, type);
                long end = System.nanoTime();
                total += end - start;
            }
            double average = total / 5.0 / 1_000_000;
            System.out.printf(
```

```
                "%s: %.3f ms | Checksum: %d%n",
                methods[type - 1], average, checksum
            );
        }
    }
    public static void main(String[] args) {
        test(1, 6);
        test(10000, 50);
        test(1000, 100);
    }
}
```



Step 6: Design Decision

For the railway ticket generation system:

- Per-ticket text: Use StringBuilder because each ticket is constructed independently, and synchronization is unnecessary.
- Shared booking log: Use StringBuffer when multiple threads directly append to the same buffer. Alternatively, use a StringBuilder protected by a suitable synchronization mechanism.

This provides efficient ticket generation and safe shared-log updates.

Step 7: Alternative Concept and Complexity Analysis

Let n represent the total number of characters in a ticket.

Method	**Time complexity**	**Extra space**
Repeated String +	(O(n^2))	(O(n^2)) cumulative intermediate allocation
StringBuilder	(O(n)) amortized	(O(n))
StringBuffer	(O(n)) amortized	(O(n))

Repeated concatenation repeatedly copies existing characters. StringBuilder and StringBuffer use expandable internal storage.

Alternative approaches:

- String.join() – useful when fields are already available as a collection; approximately \(O(n)\) for the output.
- String.format() – readable, but generally has greater formatting overhead.
- StringJoiner – useful for joining fields with delimiters.

Conclusion

The experiment demonstrates that StringBuilder is the most suitable choice for generating railway e-tickets because it provides faster text modification with fewer intermediate objects.

String is suitable for fixed text, while StringBuffer is useful when multiple threads modify a shared text buffer.

Q2 – When Is ArrayList an Appropriate Engineering Choice?

Step 1: Aim

To compare ArrayList and LinkedList for a music playlist application by analysing insertion, deletion, accessing and iteration operations.

Step 2: Concept

1. ArrayList

- Uses a dynamic array to store elements.
- Provides fast random access.
- Insertion and deletion at the beginning or middle require shifting elements.
- Uses less memory per element than LinkedList.

2. LinkedList

- Uses nodes connected through links.
- Insertion and deletion are efficient when the required node is already located.
- Random access is slow because nodes must be traversed.
- Requires extra memory for storing links.

Step 3: Design the Experiment

We compare both collections using three playlist sizes.

Test	**Number of songs**
Small	1,000
Medium	10,000
Large	100,000

Operations tested:

1. Adding songs at the end.
2. Inserting songs at the beginning.
3. Removing songs from the middle.
4. Accessing the kth song.
5. Iterating through the playlist.

Step 4: Java Program

```
import java.util.*;
public class PlaylistBenchmark {
    static void test(int n) {
        List<Integer> array = new ArrayList<>();
        List<Integer> linked = new LinkedList<>();
        long start, end;
        // Adding elements at the end
        start = System.nanoTime();
        for (int i = 0; i < n; i++)
            array.add(i);
        end = System.nanoTime();
        long arrayAdd = end - start;
        start = System.nanoTime();
        for (int i = 0; i < n; i++)
            linked.add(i);
        end = System.nanoTime();
        long linkedAdd = end - start;
        // Insertion at front
        start = System.nanoTime();
        for (int i = 0; i < 100; i++)
            array.add(0, i);
        end = System.nanoTime();
        long arrayInsert = end - start;
        start = System.nanoTime();
        for (int i = 0; i < 100; i++)
            linked.add(0, i);
        end = System.nanoTime();
        long linkedInsert = end - start;
        // Accessing middle element
        start = System.nanoTime();
```

```
        int a = array.get(n / 2);
        end = System.nanoTime();
        long arrayAccess = end - start;
        start = System.nanoTime();
        int b = linked.get(n / 2);
        end = System.nanoTime();
        long linkedAccess = end - start;
        // Iteration
        start = System.nanoTime();
        for (int x : array) { }
        end = System.nanoTime();
        long arrayIterate = end - start;
        start = System.nanoTime();
        for (int x : linked) { }
        end = System.nanoTime();
        long linkedIterate = end - start;
        System.out.println("\nPlaylist size: " + n);
        System.out.println("Operation      ArrayList(ns) LinkedList(ns)");
        System.out.println("Add end        " + arrayAdd + "       " + linkedAdd);
        System.out.println("Insert front   " + arrayInsert + "       " + linkedInsert);
        System.out.println("Access middle  " + arrayAccess + "       " + linkedAccess);
        System.out.println("Iteration      " + arrayIterate + "       " + linkedIterate);
    }
    public static void main(String[] args) {
        test(1000);
        test(10000);
        test(100000);
    }
}
```

```
Output


Playlist size: 1000
Operation      ArrayList(ns) LinkedList(ns)
Add end        108960        276820
Insert front   56220         43750
Access middle  2980          8780
Iteration      287810        65125174

Playlist size: 10000
Operation      ArrayList(ns) LinkedList(ns)
Add end        14269809       875420
Insert front   272460         38030
Access middle  1320          94850
Iteration      936120         979960

Playlist size: 100000
Operation      ArrayList(ns) LinkedList(ns)
Add end        13739609       70601873
Insert front   11981559       66870
Access middle  7840          279350
Iteration      1766770        1885790
```

Step 5: Complexity Analysis

Operation	**ArrayList**	**LinkedList**
Add at end	O(1) amortized	O(1)
Insert at front	O(n)	O(1)
Insert in middle	O(n)	O(n) to locate, O(1) to link
Remove from front	O(n)	O(1)
Access kth element	O(1)	O(n)
Iteration	O(n)	O(n)

Space complexity:

- ArrayList: O(n), with some unused array capacity.
- LinkedList: O(n), with additional memory for node references.

Step 6: Comparison and Analysis

- ArrayList performs better when frequent random access is required.
- LinkedList performs better for frequent insertions and deletions at the beginning.
- ArrayList generally uses less memory.
- LinkedList requires traversal to access a particular position.
- Both collections support sequential iteration in O(n).

Step 7: Changed Condition – Up Next Queue

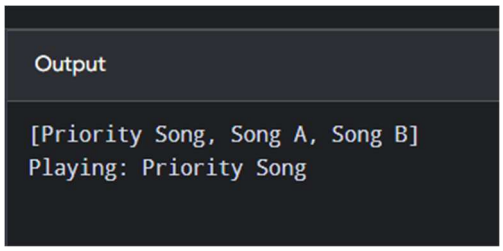
If the application introduces an "Up Next" queue where songs are frequently added at both ends and removed from the front, ArrayDeque becomes a better choice.

Reason:

- Insertion at both ends: O(1) amortized.
- Removal from both ends: O(1) amortized.
- More memory-efficient than LinkedList in typical use.
- Does not support direct indexed access.

Example:

```
import java.util.*;
public class UpNext {
   public static void main(String[] args) {
      Deque<String> queue = new ArrayDeque<>();
      queue.addLast("Song A");
      queue.addLast("Song B");
      queue.addFirst("Priority Song");
      System.out.println(queue);
      System.out.println("Playing: " + queue.removeFirst());
   }
}
```



Conclusion

ArrayList is a suitable engineering choice for a music playlist when users frequently access songs by index, add songs at the end and iterate through the playlist.

However, if frequent insertion and deletion at both ends are required, ArrayDeque is a better choice for the Up Next queue.

Q3 – Engineer the hashCode–equals Contract

Step 1: Aim

To investigate how the equals() and hashCode() methods work together in Java HashSet and understand how incorrect implementation leads to duplicate entries.

Step 2: Concept

1. equals() method

- Compares two objects based on their contents.
- Returns true if both objects are logically equal.
- In this problem, students are considered equal if their registration numbers are equal.

2. hashCode() method

- Returns an integer hash value for an object.
- HashSet uses this value to identify the bucket where an object should be stored.
- Objects that are equal must have the same hash code.

HashCode–equals contract:

1. If two objects are equal using equals(), their hash codes must be equal.
2. If two objects have equal hash codes, they need not be equal.
3. The hash code should remain consistent while equality-related fields remain unchanged.

Step 3: Problem Analysis

The given Student class overrides equals() but does not override hashCode().

Therefore, two students with the same registration number may have different hash codes.

HashSet may store both objects, creating duplicate entries.

Example:

Student s1 = new Student("192311001", "Arun");

Student s2 = new Student("192311001", "Arun");

Both students are logically equal, but without a proper hashCode() implementation, HashSet may treat them as different entries.

Step 4: Test Cases and Predictions

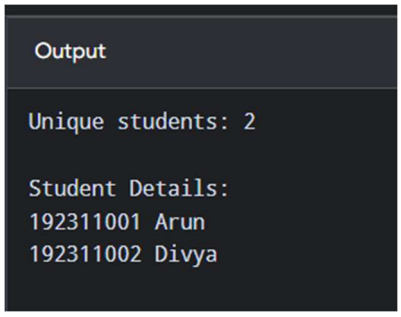
Test	**Input**	**Expected set size**
Test 1	Different registration numbers	2
Test 2	Same registration number	1

Test 3	Same registration number scanned repeatedly	1

Step 5: Complete Java Program

```
import java.util.HashSet;
import java.util.Set;
import java.util.Objects;
class Student {
    private final String regNo;
    private final String name;
    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return regNo.equals(s.regNo);
    }
    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }
    @Override
    public String toString() {
        return regNo + " " + name;
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));
        System.out.println("Unique students: " + present.size());
        System.out.println("\nStudent Details:");
        for (Student s : present) {
            System.out.println(s);
        }
    }
}
```



Why Do Duplicates Appear?

HashSet internally uses hashCode() and equals().

1. First, HashSet calculates the object's hash code.
2. It identifies the appropriate bucket.
3. It compares objects in that bucket using equals().
4. If an equal object exists, the new object is not inserted.
5. Otherwise, it inserts the object.

When hashCode() is not overridden, logically equal objects may go into different buckets, allowing duplicates.

Alternative Concept and Complexity Analysis

We can also use TreeSet or HashMap.

Collection	**Insertion / lookup**	**Duplicate identification**
HashSet	O(1) average	hashCode() and equals()
TreeSet	O(log n)	compareTo() or Comparator

HashMap	O(1) average	Key hashCode() and equals()

Conclusion

The experiment demonstrates that overriding both equals() and hashCode() is essential when storing custom objects in HashSet.

By using registration number as the equality field and generating the hash code from the same field, duplicate student entries are prevented.

Q4 – Choosing a Map for a Pharmacy Inventory

Step 1: Aim

To compare HashMap, TreeMap and LinkedHashMap for managing hospital pharmacy inventory based on insertion, searching, retrieval, iteration and ordering.

Step 2: Concept

1. HashMap

- Stores data in key-value pairs.
- Provides O(1) average insertion and searching.
- Does not maintain insertion or sorted order.
- Suitable for fast medicine lookup.

2. TreeMap

- Stores keys in sorted order.
- Provides O(log n) insertion and searching.
- Automatically maintains medicine codes in ascending order.
- Suitable for generating sorted reports.

3. LinkedHashMap

- Maintains insertion order.
- Provides O(1) average insertion and searching.
- Useful for displaying medicines in the order they were added.

Step 3: Comparison Before Implementation

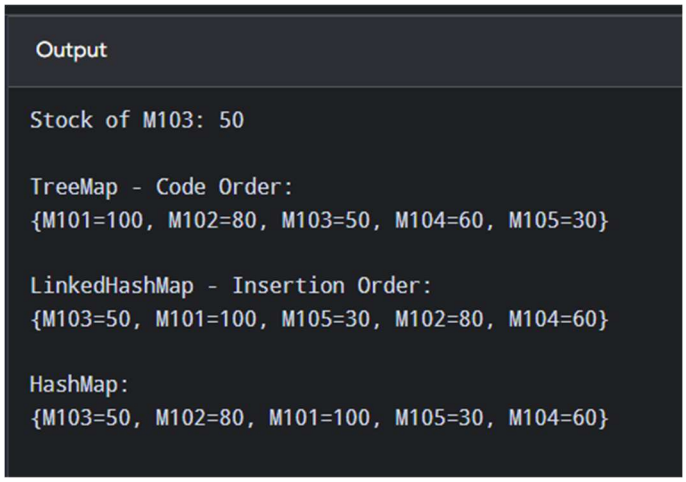
Operation	**HashMap**	**TreeMap**	**LinkedHashMap**
put()	O(1) average	O(log n)	O(1) average
get()	O(1) average	O(log n)	O(1) average
Iteration	O(n)	O(n)	O(n)
Sorted order	No	Yes	No

Insertion order	No	No	Yes

Step 4: Java Implementation

```
import java.util.*;
public class PharmacyInventory {
    public static void main(String[] args) {
        Map<String, Integer> hashMap = new HashMap<>();
        Map<String, Integer> treeMap = new TreeMap<>();
        Map<String, Integer> linkedMap = new LinkedHashMap<>();
        String[] codes = {
            "M103", "M101", "M105", "M102", "M104"
        };
        int[] stock = {50, 100, 30, 80, 60};
        // Insert medicines
        for (int i = 0; i < codes.length; i++) {
            hashMap.put(codes[i], stock[i]);
            treeMap.put(codes[i], stock[i]);
            linkedMap.put(codes[i], stock[i]);
        }
        // Lookup
        System.out.println("Stock of M103: "
                        + hashMap.get("M103"));
        // Code order report
        System.out.println("\nTreeMap - Code Order:");
        System.out.println(treeMap);
        // Insertion order report
        System.out.println("\nLinkedHashMap - Insertion Order:");
        System.out.println(linkedMap);
        // HashMap report
        System.out.println("\nHashMap:");
        System.out.println(hashMap);
    }
```

}



Scalability Analysis

For a hospital with 100,000 medicines:

- HashMap provides fast average lookup.
- TreeMap maintains sorted order without additional sorting.
- LinkedHashMap maintains insertion order without additional sorting.

My deployment decision: Use LinkedHashMap as the primary inventory structure if both reports are frequently required and fast lookup is important. It provides insertion order and average O(1) lookup.

If code-order reporting is more frequent, TreeMap is preferable.

Alternative Concept – Sorted Array

If inventory is stored in a sorted array and searched using Arrays.binarySearch():

Operation	**Sorted Array**
Lookup	O(log n)
Insertion	O(n)
Deletion	O(n)
Code-order report	O(n)
Space	O(n)

Comparison:

- HashMap: Faster average lookup, but no automatic ordering.
- TreeMap: O(log n) lookup and automatic sorting.
- Sorted array: O(log n) lookup, but insertion requires shifting elements.

Why Is Hashtable Not Preferred in New Code?

- Hashtable is a legacy collection.
- Its methods are synchronized.
- It does not allow null keys or null values.
- Modern alternatives provide more flexibility.
- For concurrent access, ConcurrentHashMap is generally preferred.

Conclusion

The experiment shows that HashMap, TreeMap and LinkedHashMap have different advantages.

HashMap is suitable for fast lookup, TreeMap is suitable for sorted reports, and LinkedHashMap is suitable for maintaining insertion order.

Q5 – Derive a Generic Ranking Component from a Principle

Step 1: Aim

To develop a reusable Java ranking component using Generics and Comparator to rank teams based on points, goal difference and team name.

Step 2: Concept

1. Comparable

- Used to define the natural ordering of objects.
- Implemented inside the class.
- Uses the compareTo() method.

2. Comparator

- Used to define custom ordering.
- Implemented separately from the class.
- Uses the compare() method.
- Supports multiple sorting conditions.

3. Generics

- Allows a class or method to work with different data types.
- Provides type safety.
- Reduces unnecessary type casting.

4. Wildcards

List<? extends T> allows a method to accept a list containing T or its subclasses.

Step 3: Ranking Rules

The sports leaderboard follows three rules:

1. Higher points should be ranked first.
2. If points are equal, higher goal difference should be ranked first.
3. If both are equal, team names should be arranged alphabetically.

Example:

Team	**Points**	**Goal Difference**
Panthers	9	4
Eagles	9	6
Cobras	6	1
Bisons	9	4

Step 4: Java Implementation

```
import java.util.*;

class Team {

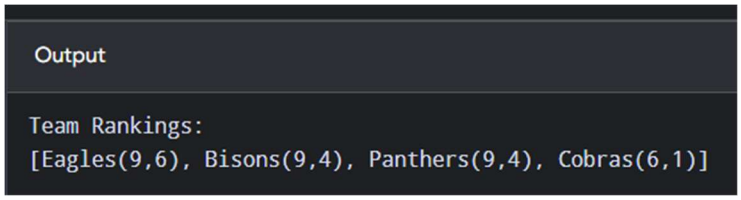
    final String name;

    final int points;
```

```
    final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }
    @Override
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }
    void add(T item) {
        items.add(item);
```

```
    }
    List<T> ranking() {
        List<T> result = new ArrayList<>(items);
        result.sort(order);
        return result;
    }
}
public class Main {
    public static void main(String[] args) {
        // Ranking: Points, Goal Difference, Team Name
        Comparator<Team> byRank = (a, b) -> {
            if (a.points != b.points) {
                return Integer.compare(b.points, a.points);
            }
            if (a.goalDiff != b.goalDiff) {
                return Integer.compare(b.goalDiff, a.goalDiff);
            }
            return a.name.compareTo(b.name);
        };
```

```
        Leaderboard<Team> board =
            new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println("Team Rankings:");
        System.out.println(board.ranking());
    }
}
```



Explain Why Bisons Is Ranked Above Panthers

Both Bisons and Panthers have:

- Points = 9
- Goal difference = 4

Since both values are equal, the comparator checks the team names alphabetically.

Bisons comes before Panthers alphabetically.

Therefore, Bisons is ranked above Panthers.

Correctness Analysis

The comparator satisfies the ranking principle because it uses consistent comparison rules.

- First, points are compared in descending order.
- Second, goal difference is compared in descending order.
- Finally, names are compared in ascending alphabetical order.

Using Comparator.comparingInt() and thenComparing() avoids subtraction-based comparison errors and provides a consistent ordering.

Alternative Concept and Complexity Analysis

Alternative 1: Comparable

If Team implements Comparable<Team>, the ranking logic can be written inside the Team class using compareTo().

However, this makes the ranking rules part of the Team class and is less flexible when the same Team needs different ranking methods.

Alternative 2: Stable sorting

We can sort three times:

1. Sort by name alphabetically.
2. Sort by goal difference descending.
3. Sort by points descending.

This works when stable sorting is used.

Generic Reusability

The Leaderboard<T> class can be reused for:

- Sports team ranking.

- Player ranking.
- College ranking.
- Quiz team ranking.

Only the Comparator needs to change.

The wildcard Comparator<? super T> allows a comparator for T or its parent type to be supplied.

Conclusion

The experiment demonstrates that Generics and Comparator can be used to create a reusable ranking system.

The comparator correctly ranks teams based on points, goal difference and alphabetical order.

Q6 – Priority, Order and Fairness in a Queue

Step 1: Aim

To implement and compare different Java Queue implementations for a hospital emergency management system based on priority, insertion order, fairness and processing time.

Step 2: Concept

1. LinkedList

- Follows FIFO (First In, First Out).
- Patients are treated in the order they arrive.
- Supports insertion and deletion at both ends.

2. PriorityQueue

- Processes elements according to priority.
- Higher-priority patients are treated first.

- Does not guarantee FIFO ordering for patients with equal priority unless a tie-breaking rule is added.

3. ArrayDeque

- Implements a double-ended queue.
- Supports insertion and deletion at both ends.
- Suitable for normal FIFO processing.

Step 3: Queue Comparison

Feature	**LinkedList**	**PriorityQueue**	**ArrayDeque**
Ordering	FIFO	Priority-based	FIFO
Insertion	O(1) at end	O(log n)	O(1) amortized
Removal	O(1) at front	O(log n)	O(1) amortized
Searching	O(n)	O(n)	O(n)
Priority handling	No	Yes	No
Memory usage	Higher	Moderate	Generally lower

Step 4: Design the Experiment

Consider the following patients:

Patient	**Priority**	**Arrival order**
Arun	3	1
Divya	1	2
Ravi	2	3
Priya	1	4

Step 5: Java Implementation

```
import java.util.*;

class Patient {
```

```
    String name;
    int priority;
    int arrival;
    Patient(String name, int priority, int arrival) {
        this.name = name;
        this.priority = priority;
        this.arrival = arrival;
    }
    public String toString() {
        return name + "(S" + priority + ")";
    }
}
public class Main {
    public static void main(String[] args) {
        PriorityQueue<Patient> queue =
            new PriorityQueue<>(
                Comparator.comparingInt((Patient p) -> p.priority)
                .thenComparingLong(p -> p.arrival)
            );
```

```
        queue.add(new Patient("Ravi", 3, 1));
        queue.add(new Patient("Meena", 1, 2));
        queue.add(new Patient("John", 3, 3));
        queue.add(new Patient("Asha", 5, 4));
        queue.add(new Patient("Kumar", 1, 5));
        queue.add(new Patient("Lata", 2, 6));
        while (!queue.isEmpty()) {
            System.out.print(queue.poll() + " ");
        }
    }
}
```

```
Output
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

Fairness Analysis

LinkedList and ArrayDeque:

- Follow FIFO.
- Patients are treated in arrival order.
- They do not consider emergency priority.

PriorityQueue:

- Emergency patients receive preference.
- Patients with the same priority are ordered by arrival time using the comparator.
- Normal patients may wait longer when emergency patients continuously arrive.

To improve fairness, the hospital can introduce an ageing mechanism that gradually increases the priority of patients who have waited too long.

Conclusion

The experiment demonstrates that different queue implementations are suitable for different hospital requirements.

- LinkedList is suitable for FIFO processing.
- ArrayDeque is suitable for efficient normal patient queues.
- PriorityQueue is suitable for emergency-based patient processing.

Q7 – Diagnose, Redesign and Validate

Step 1: A. Diagnose the Bottleneck (2 Marks)

Selected solution: PriorityQueue used in the hospital emergency system.

The original system processes patients according to severity:

- Severity 1 – Critical
- Severity 2 – Urgent
- Severity 3 – Normal
- Severity 4 – Minor
- Severity 5 – Least serious

Identified problem: Starvation

If critical patients continuously arrive, patients with severity 4 or 5 may wait indefinitely.

For example:

Patient	Severity	Arrival
Arun	5	1
Ravi	1	2
Divya	1	3
Kumar	1	4
Priya	1	5

The PriorityQueue always serves severity 1 patients before Arun.

Bottleneck analysis:

1. Patients with lower priority may experience indefinite waiting.
2. PriorityQueue does not automatically increase the priority of waiting patients.
3. The original comparator considers severity and arrival time but ignores waiting duration.

Step 2: B. Define the Design Objective (2 Marks)

The redesigned system should:

1. Prevent patients from waiting indefinitely.
2. Maintain priority-based treatment.
3. Preserve arrival order for equal effective priorities.
4. Reduce unnecessary repeated processing.
5. Maintain correct treatment order.

Measurable objective: A patient waiting for a specified period should gradually receive higher priority, without allowing less severe patients to automatically overtake critical patients.

Step 3: C. Design the Revised Solution (4 Marks)

Proposed solution: PriorityQueue with an Ageing Mechanism.

Ageing means gradually improving a patient's priority according to waiting time.

For example, every 10 waiting time units, the patient's effective severity improves by one level, up to severity 1.

Original severity	After 10 units	After 20 units	After 30 units
5	4	3	2
4	3	2	1
3	2	1	1
2	1	1	1
1	1	1	1

Step 4: Java Implementation

```
import java.util.*;

class Patient {

    String name;

    int severity;

    long arrival;

    Patient(String name, int severity, long arrival) {

        this.name = name;

        this.severity = severity;

        this.arrival = arrival;

    }

    public String toString() {

        return name + "(S" + severity + ")";
```

```
    }
}
public class Main {
    PriorityQueue<Patient> queue;
    long clock = 0;
    final int AGING_INTERVAL = 2;
    Main() {
        queue = new PriorityQueue<>(
            Comparator.comparingInt(
                (Patient p) -> effectivePriority(p)
            ).thenComparingLong(p -> p.arrival)
        );
    }
    int effectivePriority(Patient p) {
        long waited = clock - p.arrival;
        int improvement = (int)(waited / AGING_INTERVAL);
        return Math.max(1, p.severity - improvement);
    }
    void admit(String name, int severity) {
```

```
    queue.add(new Patient(name, severity, ++clock));
}
Patient next() {
    clock++;
    List<Patient> patients = new ArrayList<>(queue);
    queue.clear();
    queue.addAll(patients);
    return queue.poll();
}
public static void main(String[] args) {
    Main er = new Main();
    er.admit("Arun", 5);
    er.admit("Ravi", 1);
    er.admit("Divya", 1);
    er.admit("Kumar", 1);
    er.admit("Priya", 2);
    System.out.println("Treatment Order:");
    Patient p;
    while ((p = er.next()) != null) {
```

```
            System.out.println(
                p + " Effective Priority: "
                + er.effectivePriority(p)
            );
        }
    }
}
```

```
Output

Treatment Order:
Ravi(S1) Effective Priority: 1
Divya(S1) Effective Priority: 1
Kumar(S1) Effective Priority: 1
Arun(S5) Effective Priority: 1
Priya(S2) Effective Priority: 1
```

Step 5: D. Predict Before Implementing (1 Mark)

Measurement	**Original PriorityQueue**	**Redesigned PriorityQueue**
Priority handling	Fixed severity	Age-adjusted severity
Equal priority	Arrival order	Arrival order
Starvation	Possible	Reduced
Processing overhead	Lower	Higher
Fairness	Limited	Improved

Advantages

- Reduces starvation.
- Improves fairness among waiting patients.
- Maintains arrival order for equal effective priorities.

- Supports dynamic priority adjustment.

Disadvantages

- Rebuilding the queue increases processing time.
- Additional processing is required before each patient is selected.
- Clinical rules must prevent ageing from overriding genuine emergencies.

Alternative improvement

For a high-volume hospital, five FIFO queues (one for each severity) with controlled promotion of waiting patients can be considered. This avoids rebuilding the entire PriorityQueue for every removal, but requires careful management of promotions.

Conclusion

The original PriorityQueue provides efficient priority-based patient processing but may cause starvation for patients with lower priority.

The redesigned system introduces an ageing mechanism that gradually improves waiting patients' effective priority. This reduces excessive waiting and improves fairness.

Although the redesigned approach requires additional processing time, it provides better waiting-time management.